

Tecnología Digital VI

Reducción de la dimensionalidad

Licenciatura en Tecnología Digital - UTDT

Aprendizaje no supervisado

Todos los métodos que vimos hasta ahora se basan en tener una *ground truth* contra la cual poder calcular una métrica de optimización o de evaluación de un modelo. Son todos modelos dentro de Supervised Learning.

En el aprendizaje no supervisado directamente no tenemos variable *target* o *label* a predecir. De todas maneras nuestros métodos son matemáticos y necesitan optimizar *algo*. La diferencia es que ese *algo* no es un feature del conjunto de datos si no que es una métrica que determina algún tipo de patrón o propiedad interesante.

Aprendizaje no supervisado

El aprendizaje no supervisado se utiliza para encontrar diferentes estructuras en los datos.

- Al no tener justamente una verdad contra la que compararse, el éxito o no de un método no supervisado suele ser mucho más subjetivo.
- En la práctica el aprendizaje no supervisado muchas veces se usa como paso previo del supervisado, como parte del análisis de datos exploratorio (EDA).
- También hay aplicaciones directas de aprendizaje no supervisado como detección de anomalías o autoencoders.
- Ya hicimos algún paso no supervisado en las clases previas, ¿dónde?

Reducción de la dimensionalidad

Existen diferentes problemas particulares en donde un enfoque no supervisado puede ser útil.

Dos de los problemas más clásicos dentro del aprendizaje no supervisado:

- Clustering: agrupar muestras de un dataset dentro de conjuntos mediante alguna noción de similaridad.
- Reducción de la Dimensionalidad: el objetivo es manipular los features de un dataset para encontrar menos features pero más significativos o interesantes.

Análisis de componentes principales

Dentro del problema de Reducción de la Dimensionalidad un modelo muy conocido es el Análisis de Componentes Principales. (PCA)

En este análisis vamos a tratar de encontrar nuevas componentes (features/columnas/dimensiones) que busquen:

- Ser menos columnas que las originales (Dimensionality Reduction).
- Que las columnas resultantes tengan una varianza alta (muchísima información).
- Que las columnas resultantes tengan una covarianza nula (información diferente).

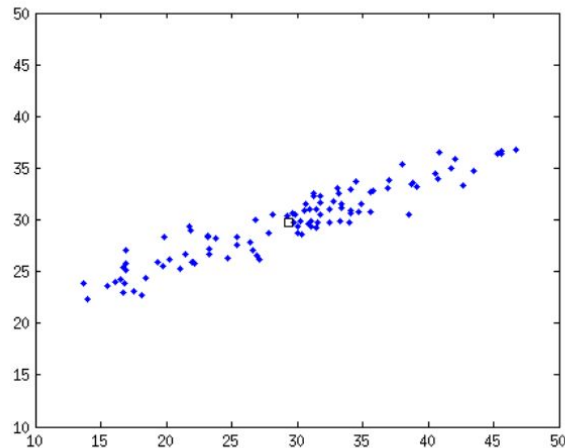
Media y varianza

$$X = \begin{bmatrix} 26.4320 & 27.7740 \\ 26.8846 & 26.5631 \\ 23.3309 & 26.6983 \\ 30.6387 & 31.5619 \\ 30.5171 & 30.8993 \\ 45.6364 & 36.6035 \\ \vdots & \vdots \\ 16.0650 & 24.0210 \end{bmatrix}$$

Media:

$$\mu = \frac{1}{n}(x^{(1)} + \dots + x^{(n)})$$

$$\mu = (29.3623, 29.7148)$$



Varianza de una variable x_k : Medida para la dispersión de los datos.

$$\sigma^2 = \frac{1}{n-1} \sum_{i=1}^n (x_k^{(i)} - \mu_k)^2$$

$$\sigma_{x_1}^2 = 66.2134, \sigma_{x_2}^2 = 12.5491$$

Covarianza

$$X = \begin{bmatrix} 26.4320 & 27.7740 \\ 26.8846 & 26.5631 \\ 23.3309 & 26.6983 \\ 30.6387 & 31.5619 \\ 30.5171 & 30.8993 \\ 45.6364 & 36.6035 \\ \vdots & \vdots \\ 16.0650 & 24.0210 \end{bmatrix}$$

Covarianza: Medida de cuánto dos variables varían de forma similar. Variables con mayor covarianza inducen la presencia de cierta dependencia o relación.

$$\sigma_{x_j x_k} = \frac{1}{n-1} \sum_{i=1}^n (x_j^{(i)} - \mu_j)(x_k^{(i)} - \mu_k)$$

No nos interesa tener dos variables tan correlacionadas: es redundante

Matriz de covarianzas

Dadas n observaciones de dos variables x_k , x_j , y $v = (1, \dots, 1)^t$:

$$\sigma_{x_j x_k} = \frac{1}{n-1} \sum_{i=1}^n (x_j^{(i)} - \mu_j)(x_k^{(i)} - \mu_k) = \frac{1}{n-1} (x_k - \mu_k v)^t (x_j - \mu_j v)$$

En la diagonal están las varianzas y la matriz es simétrica

Matriz de Covarianza:

$$X = \begin{bmatrix} 26.4320 - \mu_1 & 27.7740 - \mu_2 \\ 26.8846 - \mu_1 & 26.5631 - \mu_2 \\ 23.3309 - \mu_1 & 26.6983 - \mu_2 \\ 30.6387 - \mu_1 & 31.5619 - \mu_2 \\ 30.5171 - \mu_1 & 30.8993 - \mu_2 \\ 45.6364 - \mu_1 & 36.6035 - \mu_2 \\ \vdots & \vdots \\ 16.0650 - \mu_1 & 24.0210 - \mu_2 \end{bmatrix} \quad M_X = \frac{1}{n-1} X^t X = \begin{bmatrix} \sigma_{x_1 x_1} & \sigma_{x_1 x_2} \\ \sigma_{x_1 x_2} & \sigma_{x_2 x_2} \end{bmatrix}$$
$$= \begin{bmatrix} \sigma_{x_1}^2 & \sigma_{x_1 x_2} \\ \sigma_{x_1 x_2} & \sigma_{x_2}^2 \end{bmatrix}$$
$$M_X = \begin{bmatrix} 66.2134 & 27.1263 \\ 27.1263 & 12.5491 \end{bmatrix}$$

¿Cómo expresamos mejor nuestros datos?

Objetivo

Buscamos una transformación de los datos que disminuya la redundancia (es decir, disminuir la covarianza).

- Cambio de base: $\hat{X}^t = PX^t$ Para cambiar a un espacio donde se cumplan ciertas cosas
- ¿Cómo podemos hacerlo? Diagonalizando la matriz de covarianza. Esta matriz tiene la varianza de cada variable en la diagonal, y la covarianza en las restantes posiciones. Luego, al diagonalizar buscamos variables que tengan covarianza cero entre sí y, adicionalmente, la mayor varianza posible.

Diagonalización

En muchos casos, la presencia de autovectores–autovalores puede ser utilizada para encontrar una factorización $A = PDP^{-1}$, donde D es una matriz diagonal.

Esto significa que podemos encontrar una base donde la transformación lineal A se comporta de manera diagonal.

No toda matriz es diagonalizable pero sabemos que A (de $n \times n$) es diagonalizable si y sólo si A tiene n autovectores linealmente independiente.

Adicionalmente, si A es simétrica, entonces existe una base ortonormal de autovectores. Consecuentemente tenemos que podemos factorizar $A = PDP^t$

¿Cómo expresar mejor nuestros datos?

Cambio de base: $\hat{X}^t = PX^t$. Pasa de los datos originales a un "nuevo mundo"

Sea P ortogonal y $M_{\hat{X}}$ la matriz de covarianza de $\hat{X}$.

$$\begin{aligned}M_{\hat{X}} &= \frac{1}{n-1} \hat{X}^t \hat{X} \\&= \frac{1}{n-1} (PX^t)(XP^t) \\&= P \frac{X^t X}{n-1} P^t \\&= PM_X P^t\end{aligned}$$

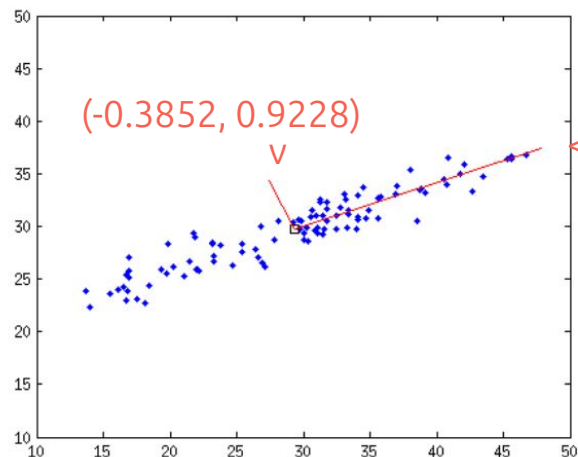
M_X es simétrica, entonces existe V ortogonal tal que $M_X = VDV^t$.

$$\begin{aligned}M_{\hat{X}} &= PM_X P^t \\&= P(VDV^t)P^t \quad \text{tomamos } P = V^t \\&= (V^t V)D(VV^t) = D \quad \text{Todo esto es para encontrar } P\end{aligned}$$

PCA en el ejemplo original

Las columnas de V son los autovectores de la matriz de covarianza

$$\begin{aligned} M_X &= \begin{bmatrix} 66.2134 & 27.1263 \\ 27.1263 & 12.5491 \end{bmatrix} && \text{Matriz de covarianza} \\ &= \underbrace{\begin{bmatrix} 0.9228 & -0.3852 \\ 0.3852 & 0.9228 \end{bmatrix}}_V \underbrace{\begin{bmatrix} 77.5362 & 0 \\ 0 & 1.2263 \end{bmatrix}}_{D=M_{\hat{X}}} \underbrace{\begin{bmatrix} 0.9228 & 0.3852 \\ -0.3852 & 0.9228 \end{bmatrix}}_{V^t} \end{aligned}$$



$\leftarrow (0.9228, 0.3852)$

Estos dos ejes están completamente decorrelacionados.

Los pasos:

- Tomamos un dataset de n filas y m columnas.
- Calculamos el vector μ de medias de cada variable.
- Construimos la matriz X de $n \times m$ tomando cada muestra, restando la media y dividiendo por la raíz de $n-1$. $x^{(i)} := (x^{(i)} - \mu) / \sqrt{n-1}$
- Creamos la matriz M de covarianzas a partir de $X^t X$.
- Diagonalizamos la matriz M y así obtenemos las componentes principales.
- Al estar ordenadas por su varianza, podemos tomar las primeras d componentes.

Hasta ahora usamos PCA para visualizar y eso ya es bastante interesante. Es claro que en las nuevas coordenadas podríamos realizar clusterizaciones más adecuadas o sacar conclusiones de patrones.

Veamos qué pasa si intentamos usar esta técnica para mejorar la clasificación de dígitos.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

data = pd.read_csv('./digit-recognizer/train.csv')

X = data.drop('label', axis=1)
y = data['label']

X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2, random_state=42)

knn = KNeighborsClassifier()

knn.fit(X_train, y_train)

accuracy = knn.score(X_val, y_val)
print("Validation Accuracy:", accuracy)

Validation Accuracy: 0.9648809523809524

%%time
accuracy = knn.score(X_val, y_val)

CPU times: user 41.6 s, sys: 18.2 s, total: 59.8 s
Wall time: 7.18 s
```

Al aplicar PCA previo a knn arreglamos aproximadamente un cuarto de los errores que teníamos.

De yapa, lo hicimos con un modelo que utiliza solamente 40 columnas en vez de 784 y su tiempo de inferencia es mucho menor.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.decomposition import PCA
from sklearn.neighbors import KNeighborsClassifier

data = pd.read_csv('./digit-recognizer/train.csv')

X = data.drop('label', axis=1)
y = data['label']

X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2, random_state=42)

pca = PCA(n_components=40)
X_train_pca = pca.fit_transform(X_train)

X_val_pca = pca.transform(X_val)

knn = KNeighborsClassifier()

knn.fit(X_train_pca, y_train)

accuracy = knn.score(X_val_pca, y_val)
print("Validation Accuracy:", accuracy)

Validation Accuracy: 0.9725

%%time
accuracy = knn.score(X_val_pca, y_val)

CPU times: user 1.98 s, sys: 4.01 ms, total: 1.99 s
Wall time: 287 ms
```

Random Forest

Al ser un problema de imágenes, los métodos basados en árboles no son para nada los más adecuados, pero eso no significa que no podamos usarlos.

Veamos un Random Forest.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier

data = pd.read_csv('./digit-recognizer/train.csv')

X = data.drop('label', axis=1)
y = data['label']

X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2, random_state=42)

rf = RandomForestClassifier(n_estimators=200, n_jobs=-1, random_state=4)

rf.fit(X_train, y_train)

accuracy = rf.score(X_val, y_val)
print("Validation Accuracy:", accuracy)

Validation Accuracy: 0.964047619047619
```


PCA+Random Forest

Si usamos PCA antes de entrenar el Random Forest nuestra métrica se degrada bastante. ¿Qué pasó?

Tener pocos features y justamente súper concentrar la información más importante en muy poquitas columnas va en contra de la decorrelación que buscamos en Random Forest.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.decomposition import PCA
from sklearn.ensemble import RandomForestClassifier

data = pd.read_csv('./digit-recognizer/train.csv')

X = data.drop('label', axis=1)
y = data['label']

X_train, X_val, y_train, y_val = train_test_split(X, y, test_size=0.2, random_state=42)

pca = PCA(n_components=30)
X_train_pca = pca.fit_transform(X_train)

X_val_pca = pca.transform(X_val)

rf = RandomForestClassifier(n_estimators=200, n_jobs=-1, random_state=4)

rf.fit(X_train_pca, y_train)

accuracy = rf.score(X_val_pca, y_val)
print("Validation Accuracy:", accuracy)

Validation Accuracy: 0.9473809523809524
```

PCA y sus limitaciones

Lo único que hicimos en PCA fue encontrar una transformación lineal que funcione como cambio de base. Al ser una transformación lineal, los features resultantes no pueden ser otra cosa que combinaciones lineales de los features originales. Si los patrones a capturar tienen componentes no lineales a PCA no le va a ser simple detectarlos.

Veamos un ejemplo de juguete muy simple con el dataset make moons https://scikit-learn.org/stable/modules/generated/sklearn.datasets.make_moons.html para ver qué sucede con la no-linealidad.

Otros métodos

Existen otros métodos que persiguen un objetivo parecido al de PCA pero utilizan una matemática subyacente diferente, lo que da lugar a modelos no lineales que capturan otro tipo de estructuras. Algunos ejemplos:

- t-SNE: <https://scikit-learn.org/stable/modules/generated/sklearn.manifold.TSNE.html>
- UMAP: <https://pypi.org/project/umap-learn/>
- PaCMAP: <https://pypi.org/project/pacmap/>